Keyed Text Watermarking: Milestone 1 Report

Data Security and Privacy, Fall 2026. Team: Barack NIYONKURU, Sylvain NDAYISHIMIYE , Raiyan ASHRAF.

1. Overview

We built a detector for a keyed, word-based text watermark. A secret key gives every word a color, green or red. A passage is watermarked by replacing red words with green ones, and the detector flags a passage when its fraction of green words is above a threshold. We prepared 315 public-domain passages, chose all settings on calibration passages only, fixed them, and then measured errors on evaluation passages. A detector flag shows that a text matches the keyed signal; it does not by itself prove who wrote the text.

2. Scheme and detector

2.1 Word rules (tokenization and normalization)

The same rules are used for counting passage lengths, watermarking and detection (watermark/tokenizer.py, documented in RULES.md):

- A word is a run of letters. Numbers, underscores and punctuation are not part of words.
- The text is normalized with Unicode NFC first, so an accented letter is always stored the same way.
- Every word is lowercased, so Book and book are the same word.
- Hyphens and apostrophes split words: well-known gives well, known; don't gives don, t.

2.2 Key and word colors

We generated one random 32-byte key once with `secrets.token_bytes(32)` (make_key.py). The script refuses to overwrite an existing key, and the key file is excluded from Git. A word's color is `hmac.digest(key, word, "sha256")[-1] & 1`, where 1 is green. Every occurrence of a word has the same color. We used this one key for every run and never replaced it or chose it based on scores.

2.3 Scoring

The score is the number of green word occurrences divided by the total number of word occurrences; repeated words count every time. A text is flagged only when the score is greater than the threshold, so a score equal to the threshold is not flagged. Input with no words (for example "123 !!") is rejected with an explicit error instead of a score. The detector also returns the total and distinct word counts. We tested each rule: a 6-word text with 4 green words scored 0.667; a text scoring exactly 0.60 was not flagged at threshold 0.60; and empty input raised the error.

3. Data

All passages come from 8 public-domain books from Project Gutenberg [22]. Passages are whole paragraphs cut by a script (make_passages.py, fixed random seed 2026). All 315 passages are distinct. We split them into calibration and evaluation before running the detector on any of them. Every book belongs to only one set, so no book appears in both calibration and evaluation.

Set	Books (passages from each)	Unmarked	For watermarking
Calibration	Pride and Prejudice (41), Dracula (31), Little Women (77), The Time Machine (6)	150	5
Evaluation	Alice in Wonderland (23), Frankenstein (36), Sherlock Holmes (77), Tom Sawyer (24)	150	10

Each set has 50 unmarked passages in each length group: short (30-99 words), medium (100-199) and long (200-400). All 15 watermarking passages have 100-200 words. Before any detector score was seen, passages E-W-01 to E-W-05 were labeled for query-access forgery (Attack 3); the selection method was to take the 5 evaluation watermarking passages with the lowest IDs, chosen before any scoring (recorded in summary.txt). Passage lists and sources are in data/passages/passages.csv and summary.txt.

4. Calibration

Calibration used only calibration passages and followed the four steps of the project description.

4.1 Unmarked calibration scores

We scored the 150 unmarked calibration passages (results/calibration_unmarked_scores.csv). Short passages varied the most: for example one 32-word passage scored 0.69 and one 30-word passage scored 0.30.

4.2 Practice watermarking

We tried three watermarking methods on the 5 calibration passages, with a target of 0.70.

- **v1, manual:** a person replaced red words with green synonyms using a helper script, with a stopping rule of target reached or 15 minutes. A teammate who did not edit (Sylvain) judged all 5 outputs as meaning kept and readable, without seeing scores.
- **v2, automatic:** a script replaced every red word that had a green WordNet [23] synonym or broader word (hypernym), most frequent words first, until the target. Small words were replaced with odd meanings, for example "are" became "ar" and "at" became "astatine".
- **v3, automatic:** the same as v2, but words with 3 letters or fewer are never replaced.

Passage	Before	v1 manual	v2 auto	v3 auto
C-W-01	0.409	0.504	0.705	0.644
C-W-02	0.479	0.629	0.701	0.675
C-W-03	0.427	0.530	0.701	0.590
C-W-04	0.547	0.705	0.703	0.703
C-W-05	0.407	0.512	0.701	0.593
Reached 0.70		1 of 5	5 of 5	1 of 5

The professor advised that the project focuses on watermarking and detection rather than keeping the exact meaning, and that automating substitution is better. We therefore chose the automatic method v3: it runs in seconds, gives the same output every time, and keeps more of the text readable than v2. Manual editing took 10 to 15 minutes per passage. The automatic versions changed 20 to 49 words per passage (v2) and 19 to 31 (v3); the manual edits are listed in calibration_log.csv. Every version of every practice passage is saved (watermarked/calibration/, files _v1, _v2 and _v3), with logs of scores and replaced words. The reviewer checked the five v1 outputs; the automatic v2 and v3 calibration outputs were not separately quality-checked, and the quality of v3 was checked on the evaluation outputs (Section 5.2).

4.3 Choosing the threshold

We tested 0.55, 0.60 and 0.65 from the project description, plus 0.51 (the midpoint between typical unmarked and practice scores). For each threshold we counted flagged unmarked calibration passages and unflagged v3 practice outputs (results/threshold_candidates_v3.csv, made by choose_threshold.py).

Threshold	Unmarked flagged (of 150)	v3 outputs missed (of 5)
0.51	27	0
0.55	5	0
0.60	1	2
0.65	1	3

We chose 0.60. Threshold 0.51 flags far too many unmarked passages (18%). Threshold 0.65 flags as few unmarked passages as 0.60 but misses more outputs. Between 0.55 and 0.60, 0.60 makes fewer errors in total (3 instead of 5) and wrongly flags fewer unmarked passages (1 instead of 5). We consider a false flag, which accuses an unmarked text, the more harmful error.

4.4 Fixed settings

Setting	Value
Key	one 32-byte random key, secrets/key.bin
Word rules	RULES.md, watermark/tokenizer.py
Threshold	0.60 (flag when score > 0.60)
Watermarking method	auto_watermark.py, v3 (skip words of 3 letters or fewer)
Target	0.70
Stopping rule	target reached, or no more red words with a green replacement

The target (0.70) is strictly above the threshold (0.60), so an output that reaches the target is flagged. Each replacement turns a red word green, so the score only rises and the final output is always the best one; it is kept even when it misses the target. The settings are stored in settings.py and were committed to Git before any evaluation passage was scored or watermarked. We made no design changes after evaluation.

5. Evaluation

5.1 Results

Measurement	Calculation	Result
Unmarked false-positive rate	flagged unmarked evaluation passages / 150	1 / 150 = 0.7%
Detection rate on watermarking outputs	flagged outputs / 10 attempts	10 / 10 = 100%
Usable watermarking success rate	flagged and passing the quality check / 10	0 / 10 = 0%

Passage	Before	After	Words changed	Flagged
E-W-01	0.562	0.657	10	yes
E-W-02	0.556	0.692	18	yes
E-W-03	0.438	0.654	40	yes
E-W-04	0.489	0.610	17	yes
E-W-05	0.441	0.661	26	yes
E-W-06	0.490	0.702	22	yes
E-W-07	0.469	0.667	32	yes
E-W-08	0.500	0.660	17	yes
E-W-09	0.504	0.658	18	yes
E-W-10	0.450	0.661	23	yes

Only E-W-06 reached the 0.70 target; the other nine stopped because no red word of 4 or more letters had a green replacement left. All ten still scored above 0.60 and were flagged. Words changed counts every replaced word occurrence, as recorded by the script (watermarked/auto_log_evaluation_v3.csv).

5.2 Quality check

Sylvain, who did not create the outputs, compared each original with its watermarked version before seeing any scores or flags. For all 10 outputs he found the prose readable, but the main meaning not preserved, so none counts as usable. Typical changes were "took" to "act" and "lest" to "fifty". The reviewer's knowledge of the project can influence the review: he knew how the method works and that meaning was not our focus, which may make him more forgiving of odd wording, or more alert to it. In calibration, he accepted one manual edit ("making your toilet" changed to "using the toilet") that we thought changed the meaning.

5.3 Length and repetition

Length group	Flagged	Tested

short (30-99)	1	50
medium (100-199)	0	50
long (200-400)	0	50

The only false positive is a short passage. With fewer words, a few green words change the score more. Total and distinct word counts are in results/evaluation_unmarked_scores.csv for the evaluation passages, and in data/passages/passages.csv for all 315 passages.

The most repetitive unmarked evaluation passage, E-U-124, has 331 words but only 165 distinct words, and scored 0.547. Its 10 most frequent words (the, was, he, and, to, that, i, it, of, me) fill 96 of the 331 positions (29%): 42 of them green and 54 red. One word can move the score a lot: "the" appears 15 times and is green; if it were red, the score would drop by about 0.045. In the extreme case, "the" repeated 1,000 times scored 1.0 and was flagged, and "was" repeated 1,000 times scored 0.0. Both texts have 1,000 words but only one color assignment, so a long text is not reliable when it has few distinct words.

5.4 Small samples and limits

Measurement	Observed	Tested	95% upper limit
False positives	1 (0.7%)	150	about 3.2% (4.74 / 150 for one error)
Missed watermarks	0 (0%)	10	about 30% (3 / 10 for zero errors)

Our error estimates are uncertain because the samples are small. The 150 unmarked evaluation passages come from only 4 books, and passages from the same book share vocabulary and style, so their results are correlated and the real uncertainty is larger than the table shows. The results describe 19th-century English fiction and may not hold for other kinds of text.

5.5 Annual false flags

As an example, assume an operator such as a school receives 10,000 unmarked submissions per year. At our measured false-positive rate of 1 in 150, that gives about 67 expected false flags per year.

Assumed yearly submissions	False-positive rate	Expected false flags
10,000	0.7% (measured, 1 / 150)	about 67
10,000	3.2% (95% upper limit)	about 320

The real number could be several times higher than 67, because the rate comes from only 150 passages. Our test passages are paragraphs from 19th-century novels, while real submissions (for example student essays) differ in topic, vocabulary and length. Short submissions are the most likely to be wrongly flagged, as our only false positive shows, so an operator receiving many short texts should expect more false flags.

6. Generation-time watermarking (required reading)

Kirchenbauer et al. [1] embed a watermark during generation by biasing green-token logits before sampling, without retraining the model. Our project instead modifies finished prose through keyed red-to-green word substitutions, so it must preserve an existing passage's meaning rather than select among plausible continuations. Our implementation picks green WordNet candidates without sense or grammar filtering: all ten evaluation outputs were flagged, but none preserved meaning, a limitation of our implementation rather than of substitution-based watermarking generally. For detection, the paper uses a length-dependent statistical test and discusses excluding repeated n-grams; our detector compares the green-word occurrence fraction with a fixed threshold of 0.60. Because every occurrence of a normalized word shares its color, repetition adds no independent evidence (one word repeated 1,000 times scores either 0 or 1), so the paper's statistical guarantees do not transfer directly to our detector. Fixed colors also make learned colors reusable across passages (Section 7.3.2). Fixed grouping is not inherently insecure, as unigram watermarking research shows [4], but its guarantees depend on assumptions absent from our implementation. A flag alone therefore establishes neither legitimate watermark generation nor authorship.

7. Threat model

7.1 Literature review of threat model

Text-watermarking research includes generation-time methods that bias token selection [1], fixed-group schemes [4], and lexical-substitution watermarking of finished text, the approach closest to ours [14]–[16]. These approaches balance detectability, text quality, and robustness to editing. Paraphrasing can weaken detection [2], [13], although residual watermark evidence may remain detectable [3]. Theoretical removal results depend on assumptions about quality-preserving transformations [6], while practical studies highlight imperfect quality assessment and reduced success under human review [7]. Color-aware substitution [9], watermark stealing [5], watermark learning [12], and detector-guided attacks [10], [11] further demonstrate how learned watermark behavior or detector feedback can support removal or forgery, although spoofed text can leave detectable traces [17]. Research also explores separating watermark generation from detection [18] and incorporating cryptographic signatures [19], [20]. For our project, this literature motivates evaluating attacker access, query costs, and meaning preservation together; published results do not directly establish our system's security.

7.2 Attacks by goal and access level

All cells describe proposed attacks and are predictions unless labeled "Measured". Success requires preserved meaning and readability. Removal must lower the score to 0.60 or below; forgery must raise it above 0.60 without using the legitimate watermarking service.

Goal	Aspect	No detector access	Detector-query access	Key access
Removal	**Strategy**	Blind synonym substitution or paraphrasing. Character changes, such as look-alike or zero-width characters [8], are alternatives.	Use detector feedback to guide substitutions. If accepted, repeated-word probes can identify red alternatives (Measured, Section 5.3); red-word insertion is another option.	Compute colors locally and replace green occurrences with suitable red alternatives. Context-aware candidate generation may help.
	Queries and effort	Zero detector queries. About 7 to 8 words must change in a 120-word output scoring 0.66, but the attacker cannot verify removal, so more rewriting and review are likely.	About 10 to 20 queries with numeric scores (one baseline plus one per tested swap; some swaps do not help); more with yes/no answers, because colors must be learned first. Editing and review remain necessary.	Zero required detector queries; optional final verification. About 7 to 8 local swaps for a 120-word output at 0.66, and only about 2 for E-W-04 (0.610); review still costs time.
	Quality risk	Changed facts, nuance, or readability.	Unsuitable replacements or unnecessary additions.	Opposite-color words may change meaning or grammar.
	Preventing condition / Defense	**Preventing condition:** Too few suitable alternatives or too much retained green vocabulary.	**Defense:** Query budgets and repetitive-input restrictions may constrain learning.	**Preventing condition:** No acceptable red alternatives. Key secrecy prevents this access level; rate limits cannot stop offline search after disclosure.
Forgery	**Strategy**	Blind rewriting may succeed by chance. If watermarked text is public, infer likely green words and use suitable replacements.	Probe source words to identify red ones (Measured, Section 5.3), learn green replacements, and substitute them. Numeric scores also support direct candidate testing.	Compute colors locally and replace red occurrences with suitable green alternatives, independently of the legitimate service.
	Queries and effort	Zero detector queries. About 12 to 13 words must change in a 120-word original scoring 0.50; corpus analysis, editing, and review may require substantial effort.	One accepted query per word probed, plus a few passage checks: for about 12 to 25 learned words, roughly 15 to 30 queries. Candidate testing and review add effort.	Zero required detector queries; optional verification. Measured: our v3 script computed colors locally and changed 10 to 40 words per evaluation passage in seconds (Section 5.1). Review remains necessary.

Goal	Aspect	No detector access	Detector-query access	Key access
	Quality risk	Inferred words may not fit; appended repetition may fail readability.	Green replacements may distort meaning.	A higher score does not establish usable prose.
	Preventing condition / Defense	**Preventing condition:** Inaccurate color inference or insufficient suitable alternatives. Reliable success is unestablished.	**Defense:** Query budgets and repetitive-input restrictions may constrain learning. Binary responses alone do not block accepted repeated-word probes.	**Preventing condition:** No acceptable green alternatives. Protect the generation key; signature-based designs offer a separate approach to verification.

Query costs are conditional estimates, not measured success budgets. Words to change ≈ |score − 0.60| × passage length, since each swap changes the score by 1 / passage length. Rejected requests must also be recorded. Character changes require independent quality review; visual similarity alone does not establish success.

7.3 Design questions

7.3.1 Key distribution

Our scheme uses the same secret key for watermarking and local detection. Anyone holding that key can calculate every word's color and select replacements for removal or forgery, so distributing the detection key also distributes the ability to construct the watermark signal, although suitable, meaning-preserving replacements are not guaranteed. We recommend keeping the key with the trusted operator and authorized watermarking process; ordinary users should query a controlled interface without receiving the key. Splitting the key so that a public detector holds only part of the signal is a possible middle ground [21]. Authentication, permissions, and query budgets can restrict access but cannot block accepted repeated-word probes (Section 7.2). The operator's detection service sees each submitted passage in plaintext; omitting text from logs reduces retention but not this access. To review a disputed result, a trusted independent reviewer needs the exact submitted text, detector version, normalization rules, threshold, and key identifier, and can then reproduce detection in a controlled environment with the correct key. This verifies the calculation, not authorship; false positives, copying, and forgery remain possible.

7.3.2 Context dependence

Our implementation assigns each normalized word a fixed color, so learned colors can be reused across passages, and rearranging the same word occurrences leaves the score unchanged. In Kirchenbauer et al.'s construction, a token's green list depends on the preceding token [1]. With context-dependent colors, a word's color can differ between contexts, so our attacks would need to learn or calculate colors per context. A substitution may also change the next token's green list, so one edit can affect multiple detection contributions. This can aid removal through disruption but makes both removal and forgery less predictable. Our word-count estimate in Section 7.2 would no longer apply directly. Context dependence alone does not eliminate repetition-based leakage. Repeating one word repeats the same word–context pair, so a probe could still reveal that pair's color, though not a globally reusable one. Robustness therefore depends on the context rule, repeated-pattern handling, and detector interface, not simply on adding context.

7.3.3 Digital signatures

Our watermark is a statistical match to a secret-key word-color pattern. It may survive some edits, but a flag proves neither legitimate generation nor authorship. A conventional digital signature binds a specific document representation to a signing key. Verification uses a public key, which does not itself enable signing. Any change to the signed representation invalidates the signature unless it is signed again. If the public key is trusted to belong to its owner, verification supports document integrity and signing-key provenance, but not human authorship or the truth of the content. Key compromise affects both approaches: a leaked watermark key enables targeted color selection, and a leaked signing private key enables forged signatures, requiring revocation, reassessment of affected records, and trustworthy timing evidence. The approaches can complement each other: signatures authenticate an original document, while watermarks may provide evidence in edited copies. Signature-based publicly detectable watermarks also exist [19], [20], but their guarantees do not automatically apply to our HMAC word-substitution scheme.

References

[1] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in Proceedings of the 40th International Conference on Machine Learning, PMLR, vol. 202, pp. 17061–17084, 2023.

[2] K. Krishna, Y. Song, M. Karpinska, J. Wieting, and M. Iyyer, “Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense,” in Advances in Neural Information Processing Systems, vol. 36, 2023.

[3] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, “On the reliability of watermarks for large language models,” in International Conference on Learning Representations, 2024.

[4] X. Zhao, P. Ananth, L. Li, and Y.-X. Wang, “Provable robust watermarking for AI-generated text,” in International Conference on Learning Representations, 2024.

[5] N. Jovanović, R. Staab, and M. Vechev, “Watermark stealing in large language models,” in Proceedings of the 41st International Conference on Machine Learning, PMLR, vol. 235, pp. 22570–22593, 2024.

[6] H. Zhang, B. L. Edelman, D. Francati, D. Venturi, G. Ateniese, and B. Barak, “Watermarks in the sand: Impossibility of strong watermarking for language models,” in Proceedings of the 41st International Conference on Machine Learning, PMLR, vol. 235, pp. 58851–58880, 2024.

[7] F. Y. Harel-Canada, B. Erol, C. Choi, J. Liu, G. J. Song, N. Peng, and A. Sahai, “Sandcastles in the storm: Revisiting the (im)possibility of strong watermarking,” in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 29698–29735, 2025, doi: 10.18653/v1/2025.acl-long.1436.

[8] N. Boucher, I. Shumailov, R. Anderson, and N. Papernot, “Bad characters: Imperceptible NLP attacks,” in 43rd IEEE Symposium on Security and Privacy, 2022.

[9] Q. Wu and V. Chandrasekaran, “Bypassing LLM watermarks with color-aware substitutions,” in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8549–8581, 2024, doi: 10.18653/v1/2024.acl-long.464.

[10] Z. Shi, Y. Wang, F. Yin, X. Chen, K.-W. Chang, and C.-J. Hsieh, “Red teaming language model detectors with language models,” Transactions of the Association for Computational Linguistics, vol. 12, pp. 174–189, 2024, doi: 10.1162/tacl_a_00639.

[11] Q. Pang, S. Hu, W. Zheng, and V. Smith, “No free lunch in LLM watermarking: Trade-offs in watermarking design choices,” in Advances in Neural Information Processing Systems, vol. 37, 2024.

[12] C. Gu, X. L. Li, P. Liang, and T. Hashimoto, “On the learnability of watermarks for language models,” in International Conference on Learning Representations, 2024.

[13] V. S. Sadasivan, A. Kumar, S. Balasubramanian, W. Wang, and S. Feizi, “Can AI-generated text be reliably detected? Stress testing AI text detectors under various attacks,” Transactions on Machine Learning Research, 2025.

[14] X. Yang, K. Chen, W. Zhang, C. Liu, Y. Qi, J. Zhang, H. Fang, and N. Yu, “Watermarking text generated by black-box language models,” arXiv preprint arXiv:2305.08883, 2023.

[15] X. Yang, J. Zhang, K. Chen, W. Zhang, Z. Ma, F. Wang, and N. Yu, “Tracing text provenance via context-aware lexical substitution,” in Proceedings of the AAAI Conference on Artificial Intelligence, 2022.

[16] U. Topkara, M. Topkara, and M. J. Atallah, “The hiding virtues of ambiguity: Quantifiably resilient watermarking of natural language text through synonym substitutions,” in Proceedings of the ACM Multimedia and Security Workshop, Geneva, Switzerland, 2006.

[17] T. Gloaguen, N. Jovanović, R. Staab, and M. Vechev, “Discovering spoofing attempts on language model watermarks,” in Proceedings of the 42nd International Conference on Machine Learning, PMLR, vol. 267, pp. 19562–19584, 2025.

[18] A. Liu, L. Pan, X. Hu, S. Li, L. Wen, I. King, and P. S. Yu, “An unforgeable publicly verifiable watermark for large language models,” in International Conference on Learning Representations, 2024.

[19] J. Fairoze, S. Garg, S. Jha, S. Mahloujifar, M. Mahmoody, and M. Wang, “Publicly-detectable watermarking for language models,” IACR Communications in Cryptology, vol. 1, no. 4, 2025.

[20] T. Zhou, X. Zhao, X. Xu, and S. Ren, “Bileve: Securing text provenance in large language models against spoofing with bi-level signature,” in Advances in Neural Information Processing Systems, vol. 37, 2024.

[21] G. Milis, T. Sander, T. Souček, H. Huang, and P. Fernandez, “Could LLM watermark detection be public?,” arXiv preprint arXiv:2610.12106, 2026.

[22] Project Gutenberg. https://www.gutenberg.org (book links in the README).

[23] Princeton University. WordNet: A Lexical Database for English. Accessed through NLTK (Bird, Klein and Loper, Natural Language Processing with Python, 2009).

Contribution statement

Barack Niyonkuru: I wrote the rules that split a text into words. I wrote the code that makes the secret key, gives each word a color, and gives each text a score. I made the detector show an error when a text has no words. I did the calibration: I scored the 150 calibration passages, edited the 5 practice passages by hand, made a program that adds the watermark by itself, and chose the threshold 0.60. I saved the final settings before the evaluation. I did the evaluation on the 150 evaluation passages and the 10 watermarked passages. I wrote the notes, the README steps, and the first draft of Sections 1 to 6 of this report. I also checked and merged the team’s changes on GitHub. My detector code was first added in commit 9fd7fa4, and my evaluation work in commit 9cb2f70.

Relevant commits:
https://github.com/Barack1995/keyed-watermarking-project/commit/6dbf59d
https://github.com/Barack1995/keyed-watermarking-project/commit/5528012
https://github.com/Barack1995/keyed-watermarking-project/commit/09b7b17
https://github.com/Barack1995/keyed-watermarking-project/commit/db26c7f
https://github.com/Barack1995/keyed-watermarking-project/commit/933cd42
https://github.com/Barack1995/keyed-watermarking-project/commit/7cd71f1

Sylvain Ndayishimiye: I found the eight free books on Project Gutenberg and made the list of books. I made the 315 passages with make_passages.py and split them into the calibration and evaluation sets. I kept each book in only one set. I chose the 5 passages for Attack 3 before any scoring. I checked the quality of the watermarked passages: I compared each one with its original, without seeing the scores, and wrote down if the meaning stayed the same and if it was easy to read. I also added the evaluation files to GitHub.

Relevant commits:
https://github.com/Barack1995/keyed-watermarking-project/commit/7b5bec8
https://github.com/Barack1995/keyed-watermarking-project/commit/fbc21d5
https://github.com/Barack1995/keyed-watermarking-project/commit/9cb2f70

Raiyan Ashraf: Developed a literature-backed security analysis drawing on 21 research papers, connecting published watermarking methods and attacks to the team’s implementation. Compared Kirchenbauer et al.’s generation-time watermarking with the project’s word-substitution scheme, examining embedding mechanisms, detection statistics, text quality, repetition, and security implications. Assessed nine technique families: blind paraphrasing, detector-guided substitution, key-guided substitution, learning from public watermarked text, repeated-word color probing, look-alike character substitution, zero-width splitting, insertion-based dilution, and guessed-word repetition. Organized these techniques into the six required threat scenarios, analyzing attacker capabilities, expected query costs, editing effort, quality risks, and defenses. Prepared the comparative literature reviews, submission threat table, and design discussions covering key distribution, context dependence, and digital signatures. Distinguished literature findings, implementation-derived calculations, and untested predictions to keep the analysis evidence-based. Discussed attack strategies and design trade-offs with teammates, provided feedback on the shared report, and helped connect the literature findings with the team’s implementation and evaluation results.

Relevant commits:
https://github.com/Barack1995/keyed-watermarking-project/tree/raiyan/submission_materials
https://github.com/Barack1995/keyed-watermarking-project/commit/9fd7fa4